

AQI 3-Day Forecasting System

Project Report

Prepared for: Project documentation / handover

Scope: End-to-end pipeline — data collection, feature engineering, feature storage, model development, MLOps lifecycle, and the user-facing forecasting application

Table of Contents

1. Executive Summary
2. System Architecture Overview
3. Data Collection
4. Feature Engineering
5. Feature Store
6. Modeling — Progression of Approaches
7. Training Methodology
8. Model Lifecycle & MLOps
9. Serving Layer — Streamlit Application

1. Executive Summary

This project builds a production-style Air Quality Index (AQI) forecasting system that predicts the average AQI for each of the next 3 days using the previous 72 hours of weather and pollutant data. The system is not a single notebook experiment — it is a full pipeline covering live data ingestion, a persistent feature store, multiple modeling approaches of increasing sophistication, an automated weekly retraining and model-promotion workflow, and a Streamlit web application that serves forecasts with confidence indicators and explainability.

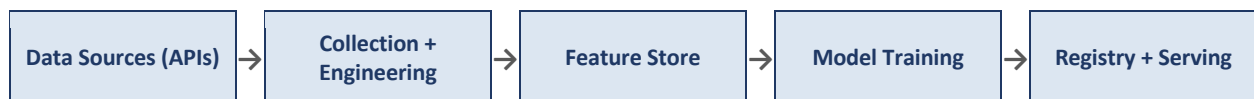
Over the course of development, the modeling approach progressed through five distinct stages, each addressing a specific limitation of the one before it:

- A classical ML ensemble (Random Forest, Extra Trees, Gradient Boosting) enhanced with handcrafted statistical features and data-driven ensemble weighting

This report documents the logic behind every stage, in the order the pipeline actually runs — from raw API data to a served forecast.

2. System Architecture Overview

At a high level, the system is organized into five layers:



Each layer is described in detail below, in the order data actually flows through the system.

3. Data Collection

Two independent, complementary data sources feed the pipeline, both from Open-Meteo:

3.1 Weather data (fetch_hourly_weather)

Pulls current-hour weather observations: `temperature_2m`, `relative_humidity_2m`, `surface_pressure`, `precipitation`, and `wind_speed_10m`, plus a timestamp. This is fetched fresh every hour for the live pipeline.

3.2 Air quality data (fetch_air_quality)

Pulls current-hour pollutant concentrations and the official AQI value: `pm2_5`, `pm10`, `carbon_monoxide`, `nitrogen_dioxide`, `sulphur_dioxide`, `ozone`, and `AQI` (sourced from Open-Meteo's `us_aqi` field).

3.3 Secondary forecast fetcher (fetch_forecast)

A separate fetcher pulls a 5-day weather forecast from OpenWeatherMap (temperature, humidity, pressure, cloud cover, wind, weather condition). This exists as an auxiliary/exploratory data source and is not part of the core training feature set used by the model — the 16 model features come exclusively from the two Open-Meteo endpoints above.

3.4 Historical dataset construction

For initial training, historical weather and air-quality CSVs are merged on their shared datetime column using an inner join, ensuring every row in the resulting historical dataset has both weather and pollutant readings for the same hour. Rows where either source is missing that hour are naturally dropped.

4. Feature Engineering

Feature engineering is implemented once and shared between two code paths — a batch/historical path (used for backfilling and initial training) and a live/incremental path (used every hour in production) — so that the exact same transformation logic is guaranteed to apply to both, avoiding train/serve skew.

4.1 Cyclical time encoding (`_add_time_and_cyclic_features`)

Raw calendar fields (hour of day, day of week, month) are not fed to the model directly, because their linear encoding (e.g. hour 23 vs hour 0) implies a false discontinuity — hour 23 and hour 0 are actually adjacent in time. Instead, hour and month are each encoded as a point on a circle using sine/cosine pairs:

- $\text{hour_sin} = \sin(2\pi \cdot \text{hour} / 24)$, $\text{hour_cos} = \cos(2\pi \cdot \text{hour} / 24)$
- $\text{month_sin} = \sin(2\pi \cdot \text{month} / 12)$, $\text{month_cos} = \cos(2\pi \cdot \text{month} / 12)$

This lets the model learn smooth periodic effects (e.g. rush-hour pollution peaks, seasonal AQI patterns) without an artificial jump at the wraparound point. The raw hour, dayofweek, and month columns are dropped afterward since only their cyclic encodings are used as model inputs.

4.2 Historical (batch) feature engineering (`engineer_historical_features`)

Given the full raw historical CSV, this path parses datetime, sorts the dataframe chronologically (critical — downstream sequence-building assumes strict time order), and applies the shared cyclic-feature function across the entire history in one pass.

4.3 Live feature engineering (`engineer_live_features`)

Given a single hour's weather response and a single hour's air-quality response, this path assembles them into a one-row dataframe with the same 16 raw columns as the historical dataset, then applies the same cyclic-feature function. Because a single row has no history to compute a rate-of-change against, any lag/diff-style feature (such as `AQI_change_rate`, described in Section 5) must be computed downstream, using the previously stored row from the feature store — not from this function.

4.4 Daily-average AQI target construction (add_daily_avg_aqi)

This is the function that defines what the model is actually trying to predict. For every hour t , it computes:

$$\text{daily_avg_AQI}[t] = \text{mean}(\text{AQI}[t : t+24])$$

implemented as a 24-hour forward rolling mean that is then shifted backward by 23 hours (`.rolling(24, min_periods=24).mean().shift(-23)`). The effect of this shift is that the value stored at row t represents the average AQI over the next 24 hours starting at t , rather than the trailing 24 hours ending at t . This forward-looking definition is what makes the value usable as a supervised-learning target rather than a historical feature. Rows at the very end of the dataset that don't have a full 24-hour future window are dropped (`dropna`), since no valid target can be computed for them.

4.5 Sequence construction for supervised learning (create_3day_sequences)

This function converts the flat, hourly time series into the (X, y) format a sequence model or a flattened ML model needs:

- **Input window (X):** for each valid starting index i , the previous `sequence_length` hours (72 in the final configuration) of all 16 feature columns are taken as one training example $\rightarrow$ shape (samples, 72, 16).
- **Target (y):** three values are read from the already forward-looking `daily_avg_AQI` column at offsets `target_start`, `target_start + 24`, and `target_start + 48` hours (where `target_start = i + sequence_length`) — corresponding to Day 1, Day 2, and Day 3's daily average AQI immediately following the input window $\rightarrow$ shape (samples, 3).
- The loop bound `len(df) - sequence_length - 72` leaves a safety margin so that the furthest target lookup (`target_start + 48`) never runs past the end of the dataframe, even accounting for the 24-hour forward window already baked into `daily_avg_AQI` itself.

This single function is shared by every model in the project (deep learning, hybrid, and classical ML) — guaranteeing all approaches were trained and evaluated on identically defined inputs and targets.

5. Feature Store

Rather than recomputing features from raw CSVs on every run, engineered features are persisted to a Postgres-backed feature store (Supabase), giving the training pipeline, the weekly retrain job, and the live serving app a single shared, queryable source of truth.

5.1 Historical backfill (upload_historical_features)

The fully processed historical dataset is bulk-loaded into an `aqi_features` table in one operation (`if_exists="replace"`), establishing the initial feature history.

5.2 Hourly incremental upload (upload_features)

Called once per hour by the live pipeline, this function:

- Floors the incoming timestamp to the hour, for consistent keying.
- Checks whether the `aqi_features` table exists yet; if not, creates it from this first row (with `AQI_change_rate` initialized to 0, since there is no prior row to compare against).
- If the table exists, checks for a duplicate timestamp and skips the insert if that hour has already been recorded — making the upload idempotent and safe to re-run.
- Computes `AQI_change_rate` as the percentage change between the incoming AQI reading and the most recently stored AQI value in the table (guarding against division by zero when the previous AQI was 0).
- Appends the new row.

5.3 Reading features (`read_features`)

A simple, chronologically ordered `SELECT * FROM aqi_features` — the single read path used by both the training pipeline and the live app's "latest window" lookup.

6. Modeling — Progression of Approaches

Five modeling strategies were developed, each building on lessons from the last. All operate on the identical $(72, 16) \rightarrow (3,)$ input/output contract established in Section 4.5.

Classical ML Ensemble with Engineered Features

The final stage revisited traditional tree-based models, which train far faster than any of the neural approaches and are easier to deploy and version (a consideration that matters for the weekly-retrain production pipeline described in Section 8). Three improvements were layered on top of the original simple ensemble:

- **Feature engineering:** in addition to the fully flattened 1152-value window (72×16), the same handcrafted statistics used in Stage 3 (mean, std, last value, trend per channel) were appended, giving the trees cheap, direct access to recency and trend signals that would otherwise require many splits across 1152 unordered columns to approximate.
- **Faster, stronger boosting:** the original plain `GradientBoostingRegressor` (single-threaded, no early stopping, and impractically slow at this data scale — observed hour-plus runtimes) was replaced with `HistGradientBoostingRegressor`, which is histogram-based and 10–50× faster, with built-in early stopping.
- **Data-driven ensemble weighting:** rather than a hand-picked fixed blend (e.g. 0.4 Random Forest + 0.3 Extra Trees + 0.3 Gradient Boosting), per-day mixing weights were fit numerically by minimizing validation MSE for each forecast horizon independently, using a softmax-reparameterized optimization so the resulting weights are guaranteed non-negative and sum to one. This let each

day's forecast use whichever blend of the three base models genuinely performed best for that specific horizon, rather than one blend applied uniformly across all three days.

7. Training Methodology

7.1 Chronological splitting

All train/validation (and, in earlier iterations, test) splits were performed chronologically, without shuffling — the training set is always the earliest contiguous block of time, and validation/test sets are later blocks. This is essential for time-series forecasting: a random shuffle would let the model "see the future" during training (data leakage) and produce misleadingly optimistic offline metrics that would not hold up in live deployment. Split ratios were adjusted iteratively over the project (70/15/15, then 80/20, then 85/15) to trade off training-data volume against the size of the held-out evaluation window.

7.2 Horizon-weighted loss

Training losses for Day 1, Day 2, and Day 3 were combined with increasing weight for longer horizons (1.0 / 1.2 / 1.5), reflecting that Day 3 forecasts are both intrinsically harder and arguably more valuable to get right, since they give the most advance warning.

7.3 Per-horizon evaluation

Every model was evaluated with R^2 , MAE, and RMSE reported separately per forecast day, rather than a single pooled metric, because forecast difficulty is not uniform across the 3-day horizon and pooling would obscure exactly where a model is weak.

7.4 Diagnosing performance across splits and horizons

A recurring, expected pattern was observed across every modeling stage: accuracy degrades from Day 1 to Day 3 (e.g. R^2 dropping from the high 0.8s down to the 0.3–0.5 range), and accuracy on a held-out window further in the future is consistently a little lower than on a window closer to the training period. This was diagnosed as a combination of:

- **Concept drift** — AQI-generating conditions (seasonal patterns, weather regimes, and other slowly changing factors) shift over time, so evaluation windows further from the training period naturally look somewhat different from what the model learned.
- **Compounding horizon uncertainty** — forecasting 3 days ahead is inherently a harder problem than forecasting 1 day ahead, independent of model quality.

This was distinguished explicitly from a genuine model regression by comparing like-for-like: when the validation window's position in the timeline changed (e.g. from a 70/15/15 split to an 85/15 split), the resulting numbers on the same chronological data were shown to be consistent, confirming the pattern was a property of the data and forecast horizon rather than a bug introduced by a later change.

8. Model Lifecycle & MLOps

The training script is not a one-off script — it implements a champion/challenger promotion workflow designed for repeated, automated retraining:

8.1 Candidate training (`train_candidate`)

If a production model already exists, it is loaded, deep-copied into a new "candidate," and warm-started: its estimator count is increased and it is re-fit on the latest available training data, incrementally building on the existing forest rather than starting from scratch every time. If no production model exists yet, a fresh Random Forest is trained instead.

8.2 Promotion decision (`compare_and_promote`)

The candidate and the current production model are both evaluated on the same validation set using a configurable metric (average MAE by default; average R^2 for the scheduled weekly job). The candidate is only promoted — replacing the production model file — if it genuinely beats production on that metric; otherwise, production is left unchanged. Every decision, along with the full per-day and averaged metrics for both models, is timestamped and appended to a CSV training-history log, creating a durable audit trail of every retraining run.

8.3 Model registry via Hugging Face Hub

Rather than shipping the trained model as part of the application code, the production model artifact is stored remotely on the Hugging Face Hub. `ensure_production_model()` transparently downloads it if it isn't present locally (e.g. on a fresh deployment), and `upload_production_model_to_huggingface()` pushes a newly promoted model back up, gated behind an `HF_TOKEN` environment variable so uploads only happen in authorized environments.

8.4 Weekly retraining entrypoint (`retrain_weekly`)

This function ties the above together into a single scheduled job: ensure a production model is available, pull the freshest features from the feature store, rebuild sequences, re-split chronologically, train a candidate, run the promotion decision, and report the winner — designed to be triggered by an external scheduler (e.g. a cron job or CI pipeline), with a thin wrapper script providing basic exception handling for unattended runs.

9. Serving Layer — Streamlit Application

The user-facing application closes the loop from stored features to a forecast a person can actually read and act on:

- **Self-healing model loading:** if the production model isn't present locally, it is downloaded from Hugging Face Hub on first load (with progress reporting), and both the model and a SHAP TreeExplainer are cached (`st.cache_resource`) so this only happens once per app session.

- **Live inference:** the most recent 72-hour feature window is pulled from the feature store (cached hourly via `st.cache_data(ttl=3600)` to avoid re-querying the database on every page interaction), flattened, and passed to the model to produce the 3-day forecast.
- **AQI categorization and health guidance:** each day's predicted value is mapped to the standard EPA-style AQI bands (Good / Moderate / Unhealthy for Sensitive Groups / Unhealthy / Very Unhealthy / Hazardous), displayed as color-coded cards, alongside plain-language health guidance matched to the worst category across the 3-day window.
- **Uncertainty indicator:** each day's card also displays that day's typical error (MAE), pulled from the most recent entry in the training-history log — giving users a sense of forecast reliability, not just a bare number.
- **Explainability:** for any selected forecast day, a SHAP waterfall plot shows the top 10 (of 1152) input features driving that specific prediction up or down relative to the model's average output — making the Random Forest's reasoning inspectable rather than a black box.
- **Trend visualization:** an Altair chart stitches together roughly the last 14 days of actual daily-average AQI with the 3-day forecast, using a dashed line style to visually distinguish forecast from observed history.
- **Model evaluation panel:** a separate section reads the training-history CSV and displays the most recently promoted model's full per-day and average R^2 /MAE/RMSE, giving transparency into current model performance directly in the app.